

CONTEXTGRAPH: REMEMBER THE CONDITION, NOT THE CODE

Anonymous authors

Paper under double-blind review

ABSTRACT

Coding agents can repeat costly debugging mistakes even when useful lessons exist in other repositories. Reusing this experience requires preserving transferable decisions, connecting related lessons, and retrieving them when they can guide the next action. We propose ContextGraph, an experience graph for self-improving coding agents. It distills strategies and warnings together with their applicability conditions and intended outcomes, retaining links to the source experience. The agent selects guidance using its full context before planning and searches again after errors to revise its diagnosis. After each task, successful and failed trajectories contribute new experience that subsequent tasks can immediately retrieve. ContextGraph achieves resolution rates of 69.2%, 35.1%, and 72.1% on SWE-bench Verified, SWE-ContextBench Related-Lite, and Datacurve DeepSWE, improving over no memory by 7.6, 13.7, and 11.3 percentage points, respectively, and leads the evaluated methods on all three benchmarks. Every initial memory pool in the main comparison is audited to exclude the evaluated tasks. With Gemini 3.1 Pro, ContextGraph raises Verified pass@1 from 82.3% to 88.9%. Ablations show that abstraction and agent-directed selection over the graph account for most of the gain, and that the benefit of online updates concentrates in later tasks of the stream. Most selected lessons come from other repositories, and cross-repository experience alone accounts for about half of the gain on two of the three benchmarks.

1 INTRODUCTION

Each repair produces experience in locating faults, revising code, and checking behavior. Memory reuses this experience on later tasks, offering a practical route toward recursive self-improvement (RSI) (Yin et al., 2025; Zhang et al., 2026a). Both successful and failed attempts contribute.

Recent agent-memory methods retain reflections and repair lessons (Shinn et al., 2023; Zhao et al., 2024) and organize this experience for reuse (Ouyang et al., 2026; Zhang et al., 2026b). To transfer across repositories, a lesson must guide investigation in unfamiliar code and identify approaches to avoid. The agent then determines the local change. Section 3 illustrates this transfer to Django from an earlier setup failure and a successful debugging attempt in two other repositories.

We therefore ask: *How can coding-agent memory preserve the conditions behind a useful decision and help an agent adapt that decision to unfamiliar code?* Our central idea is to remember when a lesson applies and what behavior it should preserve, so the agent can adapt it to the current task.

Building this memory involves three challenges. First, a lesson must retain the useful decision and its applicability conditions while abstracting away repository-specific names and code. Second, related experiences are scattered across repositories: similar words can describe different problems, while different errors can call for the same strategy. Retrieval needs these connections. Third, planning needs direction for investigation, while errors bring evidence that can change the diagnosis. Memory must inform both decisions, then retain the completed attempt for later tasks.

To address these challenges, we propose **ContextGraph**, a memory graph that turns experience across repositories into guidance for new repairs (Figure 1). *Experience abstraction* extracts strategies and warnings: what to do, what to avoid, and when each lesson applies. The graph links them to source experiences, problems, and errors. *Agent-directed selection* uses the full task context to search the experience pool and follow graph connections, selecting useful strategies across repositories. *Memory access* emphasizes two points: before planning and after an error. The first query uses the available task context; the second adds the error and recent attempt to guide diagnosis and re-

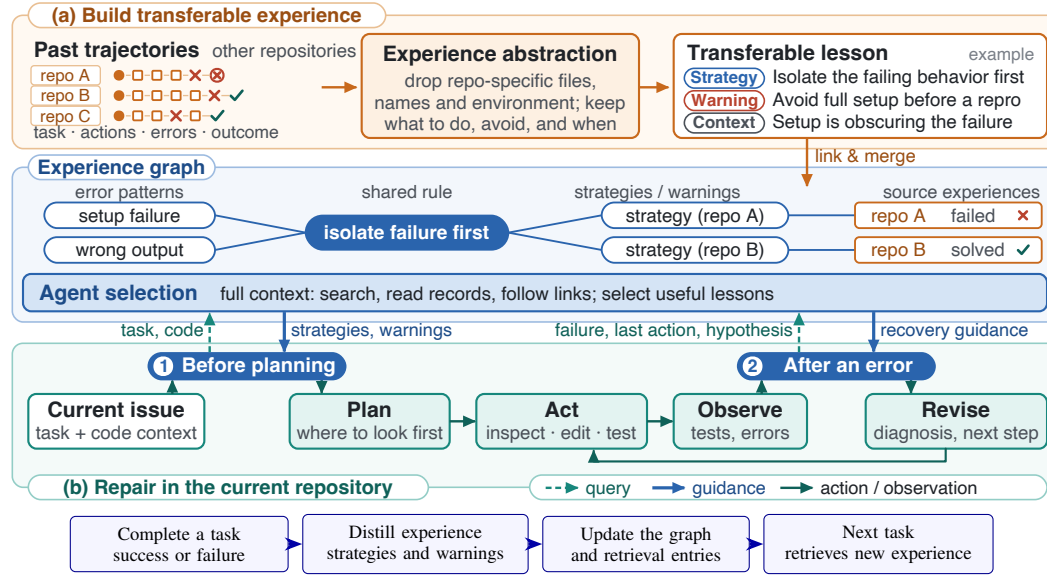


Figure 1: ContextGraph’s experience-memory architecture. Past trajectories supply strategies and warnings linked to source context. The agent retrieves guidance using its full context before planning and after errors. After every task, successful and failed trajectories update the graph; the added experience is immediately available to subsequent tasks. Graph links illustrate the organization.

sion. The agent applies these lessons through local edits and tests. After every task, ContextGraph distills the successful or failed trajectory and updates the graph, making new experience immediately available to subsequent tasks.

ContextGraph reaches 69.2% resolution on SWE-bench Verified and 72.1% on DeepSWE113, exceeding the strongest evaluated memory baselines by 2.2 and 5.9 percentage points, respectively. On Related-Lite99, its mean resolution rate is 35.1%, 13.7 points above no memory and 3.8 points above the strongest baseline (Figure 3). With Gemini 3.1 Pro and live-SWE-agent, it raises Verified pass@1 from 82.3% to 88.9%. Every initial pool of the main comparison is audited so that no evaluated task or near-duplicate fix can be selected (Appendix B).

The contributions are summarized as follows:

- We propose ContextGraph, a memory graph for reusing coding experience across repositories. It connects abstract strategies and warnings to the problems, errors, and outcomes that explain when the guidance is useful.
- We integrate memory retrieval into planning and error recovery. The full task context guides memory selection; observed failures guide subsequent searches. Successful and failed trajectories update the graph after each task, connecting current repair decisions to continuing experience accumulation.
- ContextGraph leads the evaluated methods on SWE-bench Verified, Related-Lite99, and DeepSWE113, improving Related-Lite99 by 13.7 points over no memory. Ablations show that abstraction and agent-directed selection are necessary for any gain and that graph links, online updates, and error-time search add to it; along the task stream, the benefit of online updates concentrates in later tasks. Most selected lessons come from other repositories, and they alone account for about half of the gain on two of the three benchmarks.

2 RELATED WORK

Abstracting experience for coding agents. ExpeL and ReasoningBank extract trajectory lessons (Zhao et al., 2024; Ouyang et al., 2026). SWE-Exp and ExpeRepair reuse repair experience; Mem-Govern and STAIR organize cards and plans (Chen et al., 2025; Mu et al., 2026; Wang et al., 2026; Xu et al., 2026). Memory Transfer Learning finds that abstract insights transfer across coding domains more readily than detailed traces (Kim et al., 2026). Detailed traces can instead carry mislead-

Table 1: Positioning against the memory baselines in Section 5. Organization describes how stored experience is connected; selection describes how lessons are chosen for a task. ✓: supported by the method’s description and open-source implementation; ◦: partial or not enabled in the default configuration used in our comparison; ×: absent.

Method	Memory unit / organization	Selection	After errors	Keeps failures	Online updates
ExpeL	Insights and trajectories / flat	Top- k similarity	×	✓	×
ReasoningBank	Reasoning strategies / flat	Top- k similarity	×	✓	✓
ACE	Playbook entries / sectioned playbook	Whole playbook in context	×	✓	✓
ExpeRepair	Episodes and rules / stage-indexed	Top- k per stage	✓	✓	✓
ExpGraph	Experience nodes / graph	PageRank + learned controller	×	✓	◦
Supermemory	Facts and relations / relational store	Hybrid search	×	◦	✓
ContextGraph	Strategies and warnings / typed graph	Agent-directed search	✓	✓	✓

ing implementation assumptions. ContextGraph retains strategies and warnings with their applicability context and outcomes. RETester generates tests from bug reports, MIRA retains requirements, and TRACE turns corrections into runtime checks (Wang et al., 2025; Liu et al., 2026; Zhou et al., 2026); ContextGraph returns textual guidance and leaves verification to the agent’s own tests.

Retrieving related experience. A-MEM links notes, Zep organizes temporal facts, and HippoRAG retrieves through graph traversal (Xu et al., 2025; Rasmussen et al., 2025; Gutiérrez et al., 2024). ExpGraph combines experience graphs, personalized PageRank, utility ranking, and a trained retrieval controller (Feng et al., 2026). ContextGraph connects repair strategies to recurring errors, shared rules, and source trajectories. The agent uses its full context to search these records and select useful guidance. Supermemory combines relational updates and hybrid retrieval (Daga et al., 2026); SWE-ContextBench evaluates memory-assisted repair (Zhu et al., 2026). Table 1 positions ContextGraph against the memory methods in our comparison.

Updating memory and improving agents. MemGPT and SWE-MeM manage memory access and coding histories (Packer et al., 2023; Gao et al., 2026). ACE evolves context, MemOp optimizes memory utility, and AdaRepair-Mem selects experience by repair stage (Zhang et al., 2026b; Guo et al., 2026; Luo et al., 2026). Gödel Agent and the Darwin Gödel Machine modify agent code; RRSI regularizes harness changes (Yin et al., 2025; Zhang et al., 2026a; Xia et al., 2026). AlphaEvolves evolves programs through executable evaluation; MulVul evolves prompts across generator and executor models (Novikov et al., 2025; Wu et al., 2026). ContextGraph adds lessons from successes and failures after each task and makes them immediately available for retrieval. Subsequent tasks retrieve this growing experience before planning and after errors, without updating model parameters.

3 PROBLEM AND MOTIVATION

A coding agent receives an issue and a repository and produces a patch. Its trajectory records actions, observations, and the outcome. ContextGraph builds an experience graph G from completed trajectories, retrieves guidance for each new task, and updates G afterward. The goal is to accumulate useful repair experience across tasks and repositories.

A debugging decision that transfers. In Django Trac #33018, empty query results are mishandled under negation. A failed `python-statemachine` attempt had spent its effort setting up a full Django application; its lesson was to isolate the failure before investing in framework setup. A successful `sievelib` trajectory also supported reproducing the failure before editing. Guided by these experiences, the agent traced the exception to `Exists.as_sql()`, corrected its handling, and added a regression test. The no-memory attempt made broad formatting changes without fixing the bug. The transferable content was the investigation strategy and warning; the agent derived the local implementation. Neither source repository is Django: the lesson crossed repositories, and Section 5.5 measures how often selected lessons do so.

From advice to applicable guidance. Figure 2 contrasts two memories for another Django case: a flat summary store returns one requirement sentence, whereas the agent selects from ContextGraph

a strategy, a warning, and the condition under which they apply, each linked to its source repair. The condition tells the agent when the lesson holds; the source lets it check how the lesson was applied before adapting it to the current code.

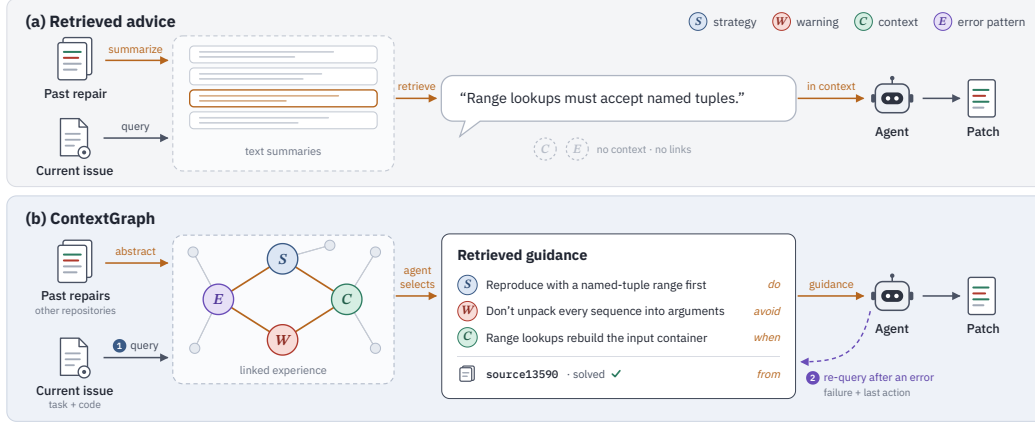


Figure 2: Retrieved advice versus ContextGraph on Django12663. (a) Flat summaries return one requirement sentence without context or source. (b) The agent selects a strategy, a warning, and their applicability context with the source experience, before planning and again after an error.

4 CONTEXTGRAPH: MEMORY FOR PLANNING, RECOVERY, AND LEARNING

ContextGraph connects experience abstraction, agent-directed selection, and memory access. The same process builds the initial graph and incorporates completed tasks (Figure 1).

4.1 EXPERIENCE ABSTRACTION: RETAIN THE USEFUL DECISION

A language model extracts lessons from the task, actions, errors, and outcome. Each lesson records an action or warning, its applicability conditions, the outcome it is meant to achieve or preserve, and source context. Abstraction removes repository-specific names while retaining the decision and its reason: the failed framework setup becomes advice to isolate the bug first.

We represent memory as a typed graph

$$G = (V, E), \quad E \subseteq V \times \mathcal{R} \times V, \quad (1)$$

where each edge has a relation type in $\mathcal{R}$. Nodes represent trajectories, fragments (action–observation segments), strategies, canonical rules (merged lessons), error patterns, problem summaries, playbook entries (retrieval-ready lessons), and communities of related fragments. The seven edge types connect a trajectory to its fragments, a strategy to its source trajectory, a fragment to an encountered error, a strategy to its canonical rule, a rule to an error it addresses, a summary to its trajectory, and a fragment to its community. These links retain both a reusable lesson and the observations that explain when it applies. Appendix C lists the node counts of a graph snapshot, and the anonymous repository (Section 5.1) releases the extraction prompts, schema, and selection code.

4.2 AGENT-DIRECTED SELECTION: FIND USEFUL EXPERIENCE

At repair step t , the agent uses its full context C_t : the issue, inspected code, actions, and failures. Memory selection is an agent decision process:

$$a_j \sim \pi_\theta(\cdot \mid C_t, H_j), \quad \mathcal{M}_t = \text{Select}_{\pi_\theta}(C_t, G_t), \quad (2)$$

where π_θ is the coding agent’s model policy, H_j records its preceding memory actions and their results, and $\mathcal{M}_t$ contains the selected lessons. Actions search the entire pool, read source records,

follow graph links, select lessons, or stop. The agent judges relevance from the returned context; there is no fixed numerical relevance score or top- k cutoff. It applies the selected guidance through local reproductions, edits, and tests (Appendix E).

4.3 MEMORY ACCESS AND UPDATES: CONNECT REPAIR TO LEARNING

Memory enters the repair loop at three points. **Before planning**, the agent searches with the full task context for investigation, reproduction, and testing strategies, and for warnings about costly approaches to avoid. **After an error**, it adds the failed action, observed error, relevant code, and current hypothesis to that context, searches again to revise its diagnosis, and uses repository tools to try the next step. **After each task**, ContextGraph distills successful and failed trajectories and links new lessons to existing experience. Updates use agent-visible observations and become immediately available to later tasks (Appendix Algorithm 1). Because memory grows during evaluation, a task late in the stream selects from a larger graph than an early one, and a retry of an unresolved task can select the lessons distilled from its own earlier failure (Section 5.4).

5 EXPERIMENTS

We ask whether ContextGraph improves repair success, which components contribute, how the on-line loop changes later tasks, how much selected experience crosses repositories, and how benefits vary across models and pool sizes.

5.1 SETUP AND EVALUATION

Tasks and metrics. We evaluate all 500 SWE-bench Verified tasks, the 99-task Related-Lite subset of SWE-ContextBench, and the 113-task Datacurve DeepSWE benchmark (Jimenez et al., 2024; Zhu et al., 2026; Huang et al., 2026). We additionally evaluate 20 Python tasks from LoLBench (Peng, 2026). A task is resolved when its patch passes the benchmark’s official evaluation. Resolution is the percentage of tasks resolved; when an evaluation run does not complete for a task, that task is excluded from the run’s denominator. $\text{pass}@k$ reports success within k attempts. Related-Lite99 and DeepSWE113 in the main comparison report means over five runs, each reset to the same initial memory pool. Table 2 gives the agents, models, and initial memory sources (SWE-agent Team, 2025; Yang et al., 2024).

Table 2: Main benchmark comparisons. The first three use DeepSeek V4 Pro 0813 with mini-SWE-agent; LoLBench uses GPT-5.5 with Codex. ContextGraph updates its memory after every task; baseline updates follow each implementation’s default behavior. No initial pool contains an evaluated task (Appendix B).

Benchmark	Tasks	Initial experience source	Reported result
Verified500	500	Nebius SWE-bench-extra trajectories	Resolution
Related-Lite99	99	SWE-bench cold-start trajectories	Five-run mean
DeepSWE113	113	LoLBench repository trajectories	Five-run mean
LoLBench Python	20	Other LoLBench Python tasks	$\text{pass}@1$, $\text{pass}@3$

Data separation and feedback. Initial memory excludes the evaluated tasks’ trajectories and reference solutions. The Nebius SWE-bench-extra trajectories that seed the Verified500 pool contain none of the 500 evaluated instances, and the same instance-level and patch-similarity audit is applied to every pool of the main comparison (Appendix B). Related-Lite99 follows the SWE-ContextBench protocol (Zhu et al., 2026): memory starts from a cold-start pool of SWE-bench trajectories and grows task by task. Tasks enter the evaluation stream in issue creation-time order. Memory updates use only agent-visible actions and observations, including its own test runs. Official test results are used for scoring only: they are neither shown to the agent nor written into memory, including between repeated attempts. Code, extraction prompts, graph schema, initial pools, and evaluation scripts are released in an anonymous repository.¹

¹<https://anonymous.4open.science/r/contextgraph-kit/>

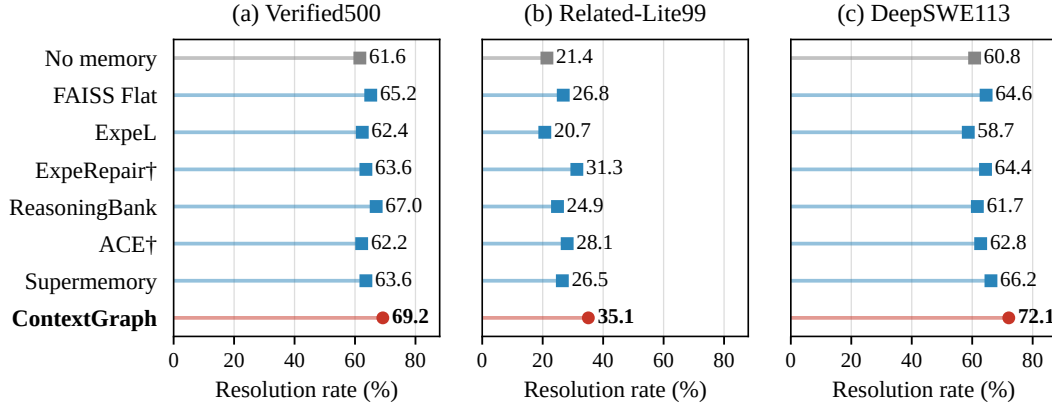


Figure 3: Repair resolution rates (%) with DeepSeek V4 Pro 0813 and mini-SWE-agent. Related-Lite99 and DeepSWE113 report five-run means. Red circles denote ContextGraph; bold values mark the best result per benchmark. † marks our adaptations to mini-SWE-agent.

Baselines. We compare no memory, FAISS Flat, ExpeL, ExpeRepair, ReasoningBank, ACE, and Supermemory (Zhao et al., 2024; Mu et al., 2026; Ouyang et al., 2026; Zhang et al., 2026b; Daga et al., 2026). FAISS Flat uses vector similarity to retrieve memory without graph traversal. We adapt ExpeRepair and ACE to mini-SWE-agent, retaining their memory procedures. We compare complete memory systems, including their construction, selection, and updates. Appendix Table 9 lists all main comparison values.

5.2 REPAIR EFFECTIVENESS

Figure 3 compares eight methods. ContextGraph reaches 69.2% on Verified500 and 72.1% on DeepSWE113, exceeding the strongest memory baselines, ReasoningBank and Supermemory, by 2.2 and 5.9 points. On Related-Lite99, its 35.1% mean improves on no memory, FAISS Flat, and the strongest memory baseline, ExpeRepair, by 13.7, 8.3, and 3.8 points, respectively. **Finding:** ContextGraph improves repair success on all three benchmarks and leads the evaluated methods on each.

Benefits with Gemini and live-SWE-agent. With Gemini 3.1 Pro and live-SWE-agent, ContextGraph raises Verified500 pass@1 from 82.3% to 88.9%, an 8.0% relative gain; pass@5 reaches 91.7% (Table 3). Without memory, additional attempts add less than one point; with memory updated after every attempt, pass@ k keeps rising, 2.8 points from pass@1 to pass@5, consistent with later attempts drawing on lessons distilled from earlier ones. On Verified500 with DeepSeek V4 Pro, ContextGraph also uses fewer tokens per task, 201K versus 236K without memory, a 14.8% reduction.

Table 3: SWE-bench Verified500 with Gemini 3.1 Pro and live-SWE-agent. Both arms follow the data-separation protocol in Section 5.1. Gains are relative improvements, (ContextGraph – No memory)/No memory, in percent.

Metric	No memory	ContextGraph	Relative gain
pass@1	82.3%	88.9%	+8.0%
pass@3	82.6%	90.4%	+9.4%
pass@5	82.8%	91.7%	+10.7%

Long-horizon development. With Codex and GPT-5.5, we build the graph from other Python tasks and evaluate 20 LoLBench tasks. All six memory baselines match no memory at 5.0% pass@1; ContextGraph reaches 15.0% pass@1 and 25.0% pass@3 (Appendix Table 15). With 20 tasks, a

two-task difference is not statistically significant, so we treat this subset as a sanity check rather than a headline result.

5.3 WHICH COMPONENTS CONTRIBUTE?

Table 4 removes one component at a time on Related-Lite99 under the main protocol. *Fixed top- k* replaces agent-directed selection with the k most similar lessons by embedding. *No graph links* keeps agent-directed search but over a flat pool of lessons without the relations in Equation 1. *Raw trajectories* skips abstraction and stores trajectory excerpts instead of strategies and warnings. *Planning-time only* disables the search after errors. *Static pool* disables updates during evaluation. *Static playbook* places the most frequently selected lessons in the system prompt of every task without memory access, testing whether the gain comes from generic advice. Each variant keeps the remaining components unchanged.

Replacing agent-directed selection with fixed top- k retrieval costs 10.8 points, the largest drop. Raw trajectories and the static playbook fall below no memory (20.5 and 20.8 versus 21.4): unabstracted traces and generic advice do not help and can mislead. Removing graph links costs 4.0 points, freezing the pool 6.4, and disabling the search after errors 1.9. **Finding:** abstraction and agent-directed selection are necessary for any gain; graph links, online updates, and error-time search each add to it.

Table 4: Ablation on Related-Lite99 with DeepSeek V4 Pro 0813 and mini-SWE-agent, five-run means. Δ is the change from the full system in percentage points.

Variant	Resolution (%)	Δ
ContextGraph (full)	35.1	—
Fixed top- k retrieval (no agent-directed selection)	24.3	−10.8
No graph links (flat lesson pool)	31.1	−4.0
Raw trajectories (no abstraction)	20.5	−14.6
Planning-time search only (no search after errors)	33.2	−1.9
Static pool (no online updates)	28.7	−6.4
Static playbook (fixed lessons, no memory access)	20.8	−14.3
No memory	21.4	−13.7

5.4 SELF-EVOLUTION ALONG THE TASK STREAM

Because every completed task is distilled into memory (Section 4.3), later tasks and retries can draw on lessons from earlier attempts in the same run. Two views expose this loop.

Retry passes. A separate Related-Lite99 experiment uses DeepSeek V4 Pro with 500 calls and \$30 per task in both conditions. The initial memory contains 291 curated repair records with reference fixes, verification hints, and history preceding the target code version. Each pass is a repair round; later passes retry only unresolved tasks, and each pass starts from the memory updated by all previous passes, including the distilled failures of the tasks being retried. In the run of Table 5, memory adds 7, 9, and 11 resolved tasks after one, two, and three passes (21, 25, and 27 versus 14, 16, and 16 of 99; five-run means in Table 6): the no-memory agent gains nothing from its third pass, whereas the agent with memory keeps converting earlier failures into repairs. Excluding ten targets with near-duplicate source fixes retains the first-pass gain, 19/89 versus 13/89 ($p = .031$). Absolute rates differ from the main comparison because this experiment uses a different budget and a curated initial pool; the paired comparison within each row is the quantity of interest. Its curated records include reference fixes, so it tests the update loop rather than the abstraction; the static-pool ablation (Table 4) shows the same effect under the main protocol. **Finding:** the gap between the two agents widens with each pass; the static-pool control in Table 6 attributes the widening to the lessons added between passes.

Position in the stream. If the online loop matters, tasks late in a run should benefit more than early ones, and the extra benefit should disappear when the pool is frozen. Table 6 tests this in the retry experiment with an additional static-pool arm, tasks arriving in issue creation-time order.

Table 5: Related-Lite99 with curated episodic memory updated after every pass. Resolved counts are cumulative. Wins count tasks solved only with memory; losses count tasks solved only without it.

Retry budget	No memory	ContextGraph	Wins/losses
1 pass	14/99	21/99	7/0
2 passes	16/99	25/99	9/0
3 passes	16/99	27/99	12/1

Without memory, each third of the stream yields the same number of first-pass repairs (4.6, 4.8, and 4.8 of 33), and a static pool adds a roughly constant amount (5.6, 6.2, and 5.6). With online updates, the first third matches the static pool (5.4 versus 5.6), while the last third resolves 8.6 tasks against 5.6 with the static pool and 4.8 without memory. Cumulative pass@ k shows the same separation widening with each pass: after three passes, online updates resolve 26.6 tasks, the static pool 20.2, and no memory 16.2. **Finding:** the benefit of online updates concentrates in later tasks and later passes, the signature of memory that improves through use rather than of a better initial pool.

Table 6: Related-Lite99 retry experiment of Table 5 (curated episodic memory, DeepSeek V4 Pro, 500 calls and \$30 per task), means over five runs in issue creation-time order. Left: tasks resolved in the first pass within each third of the stream (of 33). Right: cumulative tasks resolved (of 99) after one, two, and three passes.

Condition	Tasks 1–33	Tasks 34–66	Tasks 67–99	pass@1	pass@2	pass@3
No memory	4.6	4.8	4.8	14.2	15.8	16.2
ContextGraph, static pool	5.6	6.2	5.6	17.4	19.2	20.2
ContextGraph, online updates	5.4	7.2	8.6	21.2	24.8	26.6

5.5 CROSS-REPOSITORY TRANSFER

The Django example in Section 3 drew on two other repositories. To measure how often this happens, we log the source repository of every lesson selected in the main comparison (Table 7). Most selected lessons come from other repositories: 78.4% on Verified500, 86.2% on Related-Lite99, and 64.1% on DeepSWE113. The DeepSWE113 initial pool contains only LoLBench repositories, none of which is a DeepSWE113 repository (Appendix B), so its same-repository share comes entirely from lessons added online by earlier DeepSWE tasks. With same-repository lessons excluded, ContextGraph still improves over no memory by 1.4, 7.3, and 6.7 points; the full pool adds a further 6.2, 6.4, and 4.6 points. **Finding:** cross-repository experience alone accounts for about half of the gain on Related-Lite99 and DeepSWE113 and a smaller part on Verified500; the minority of same-repository lessons contributes disproportionately.

Table 7: Cross-repository transfer in the main comparison. Cross-repository lessons are the percentage of selected lessons whose source repository differs from the target. Other repositories only excludes same-repository lessons at selection time, including lessons added online.

Benchmark	Cross-repository lessons (%)	ContextGraph full pool	ContextGraph other repositories only	No memory
Verified500	78.4	69.2	63.0	61.6
Related-Lite99	86.2	35.1	28.7	21.4
DeepSWE113	64.1	72.1	67.5	60.8

5.6 PERFORMANCE ACROSS MODELS AND MEMORY-POOL SIZES

The experiments in this section keep the Related-Lite99 protocol of the main comparison and vary the backbone and the size of the initial pool, which holds 300 experience summaries distilled from the audited SWE-bench cold-start pool of Table 2 unless stated otherwise. The DeepSeek V4 Pro

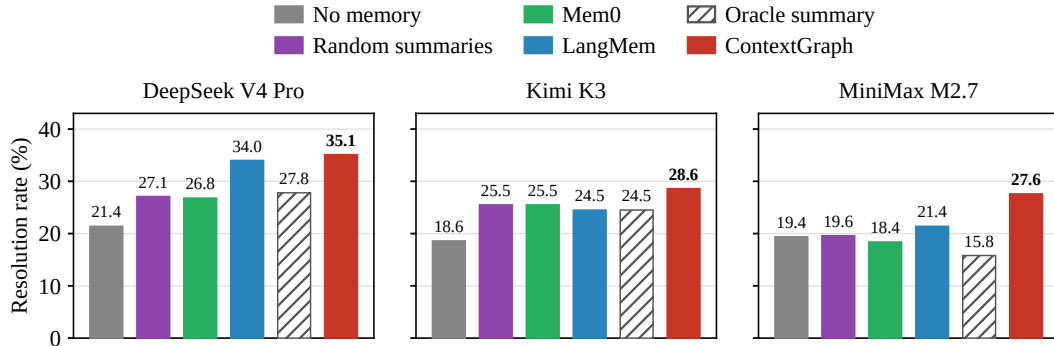


Figure 4: Related-Lite99 resolution (%) with the same 300-summary initial pool per backbone. The DeepSeek V4 Pro ContextGraph and no-memory bars are the main-comparison values and the other DeepSeek V4 Pro bars follow the same protocol; Kimi K3 and MiniMax M2.7 are single runs over completed evaluations. ContextGraph is red; the hatched oracle receives a known related summary.

ContextGraph and no-memory entries with 300 summaries are therefore the main-comparison values, and the other DeepSeek V4 Pro entries follow the same protocol; Appendices C and D give the remaining settings and further results.

Benefits across language models. Memory methods draw from the same 300-summary pool within each backbone (Figure 4), including Mem0 (Chhikara et al., 2025) and LangMem (LangChain, 2025). With DeepSeek V4 Pro, ContextGraph reaches 35.1% resolution, compared with 21.4% without memory; with Kimi K3 and MiniMax M2.7, 28.6% and 27.6% against 18.6% and 19.4%. The oracle is given one known related summary, bypassing retrieval; ContextGraph can retrieve several and exceeds the oracle on every backbone. **Finding:** ContextGraph improves resolution with all three backbones and reaches the highest rate in each comparison, whereas the other memory methods do not improve consistently (Mem0 falls below no memory on MiniMax M2.7).

Pool size. With the DeepSeek V4 Pro agent and memory-selection procedure held fixed, resolution rises from 20.2% to 35.1% as the initial pool grows from 50 to 300 summaries (Table 8). **Finding:** larger initial pools yield stronger repair.

Table 8: Related-Lite99 with different initial memory-pool sizes, DeepSeek V4 Pro. The 300-summary column is the main comparison.

Number of summaries	50	100	200	300
Resolution rate	20.2%	26.3%	28.3%	35.1%

6 CONCLUSION

ContextGraph remembers when a lesson applies and what behavior it should preserve, linking transferable strategies and warnings to their source experience. Agents select guidance before planning and after errors and contribute new experience after each task. With DeepSeek V4 Pro, it achieves 69.2%, 35.1%, and 72.1% resolution on Verified500, Related-Lite99, and DeepSWE113, leading the evaluated methods on each, and Gemini 3.1 Pro reaches 88.9% pass@1 on Verified500. Ablations attribute the gain to abstraction and agent-directed selection, online updates help more along the task stream, and most selected lessons come from other repositories. These results establish experience graphs as a strong starting point for self-improving coding-agent memory.

REFERENCES

- Silin Chen, Shaoxin Lin, Yuling Shi, Heng Lian, Xiaodong Gu, Longfei Yun, Dong Chen, Lin Cao, Jiyang Liu, Nu Xia, and Qianxiang Wang. SWE-Exp: Experience-driven software issue resolution, 2025. URL <https://arxiv.org/abs/2507.23361v2>.
- Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready AI agents with scalable long-term memory, 2025. URL <https://arxiv.org/abs/2504.19413>.
- Soham Daga, Sreeram Sreedhar, and Dhruva Shah. Supermemory is state of the art in agent memory, 2026. URL <https://supermemory.ai/research/longmembench/>. Supermemory Research, LongMemEval-S evaluation report, May 2026.
- Tao Feng, Chongrui Ye, Tianyang Luo, Jingjun Xu, Xueqiang Xu, Haozhen Zhang, Zhigang Hua, Yan Xie, Shuang Yang, Ge Liu, and Jiaxuan You. ExpGraph: Model-agnostic experience learning with graph-structured memory for LLM agents, 2026. URL <https://arxiv.org/abs/2605.30712v1>.
- Shuzheng Gao, Wenhao Zeng, Zhaojian Yu, Jianqiao Wangni, Chaozheng Wang, Kai Cai, Shilin He, and Michael R. Lyu. SWE-MeM: Learning adaptive memory management for long-horizon coding agents, 2026. URL <https://arxiv.org/abs/2606.28434v1>. Version 1.
- Xuehang Guo, Zora Zhiruo Wang, Qingyun Wang, Graham Neubig, and Xingyao Wang. Enhancing software engineering through closed-loop memory optimization, 2026. URL <https://arxiv.org/abs/2606.05646v1>.
- Bernal Jiménez Gutiérrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. HippoRAG: Neurobiologically inspired long-term memory for large language models. In *Advances in Neural Information Processing Systems*, volume 37, 2024. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/6ddc001d07ca4f319af96a3024f6dbd1-Abstract.html.
- Wenqi Huang, Charley Lee, Leonard Tng, and Serena Ge. DeepSWE: Measuring frontier coding agents on original, long-horizon engineering tasks, 2026. URL <https://arxiv.org/abs/2607.07946>.
- Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues?, 2024. URL <https://arxiv.org/abs/2310.06770>.
- Kangsan Kim, Minki Kang, Taeil Kim, Yanlai Yang, Mengye Ren, and Sung Ju Hwang. Memory transfer learning: How memories are transferred across domains in coding agents, 2026. URL <https://arxiv.org/abs/2604.14004v1>. Version 1.
- LangChain. LangMem, 2025. URL <https://github.com/langchain-ai/langmem>. Software library.
- Yining Liu, Chenyu Yang, Boyan Li, Rui Mao, and Yuyu Luo. MIRA: Evidence-verified repair memory for text-to-sql correction, 2026. URL <https://arxiv.org/abs/2608.06950v1>.
- Z. C. Luo, J. C. Guo, W. J. He, S. Y. Wang, J. C. Yu, F. M. Zhao, Y. Chen, T. Cao, L. Q. Liu, N. Zheng, W. Xu, J. Jiang, and Z. M. Zhao. AdaRepair-Mem: Adaptive experience orchestration for repository-level program repair, 2026. URL <https://arxiv.org/abs/2609.20130v1>.
- Fangwen Mu, Junjie Wang, Lin Shi, Song Wang, Shoubin Li, and Qing Wang. ExpeRepair: Dual-memory enhanced LLM-based repository-level program repair, 2026. URL <https://arxiv.org/abs/2506.10484v2>. Version 2; accepted at FSE 2026.

- Alexander Novikov, Ngán Vũ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, M. Pawan Kumar, Abigail See, Swarat Chaudhuri, George Holland, Alex Davies, Sebastian Nowozin, Pushmeet Kohli, and Matej Balog. AlphaEvolve: A coding agent for scientific and algorithmic discovery, 2025. URL <https://arxiv.org/abs/2506.13131>.
- Siru Ouyang, Jun Yan, I-Hung Hsu, Yanfei Chen, Ke Jiang, Zifeng Wang, Rujun Han, Long Le, Samira Daruki, Xiangru Tang, Vishy Tirumalashetty, George Lee, Mahsan Rofouei, Hangfei Lin, Jiawei Han, Chen-Yu Lee, and Tomas Pfister. ReasoningBank: Scaling agent self-evolving with reasoning memory. In *International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=jL7fwchScm>.
- Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems, 2023. URL <https://arxiv.org/abs/2310.08560>.
- Yun Peng. LoLBench: Long-horizon tasks on large software systems. Hugging Face dataset, 2026. URL <https://huggingface.co/datasets/lolbench26/LoLBench>.
- Preston Rasmussen, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. Zep: A temporal knowledge graph architecture for agent memory, 2025. URL <https://arxiv.org/abs/2501.13956>.
- Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 36, 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/1b44b878bb782e6954cd888628510e90-Abstract-Conference.html.
- SWE-agent Team. mini-SWE-agent: The minimal software engineering agent. Software repository, 2025. URL <https://github.com/SWE-agent/mini-swe-agent>.
- Haibo Wang, Zhuolin Xu, and Shin Hwei Tan. Testing refactoring engine via historical bug report driven llm, 2025. URL <https://arxiv.org/abs/2501.09879v2>.
- Qihao Wang, Ziming Cheng, Shuo Zhang, Fan Liu, Rui Xu, et al. MemGovern: Enhancing code agents through learning from governed human experiences, 2026. URL <https://arxiv.org/abs/2601.06789v2>.
- Zihan Wu, Jie Xu, Yun Peng, Chun Yong Chong, and Xiaohua Jia. MulVul: Retrieval-augmented multi-agent code vulnerability detection via cross-model prompt evolution. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 8660–8673, 2026. doi: 10.18653/v1/2026.acl-long.391. URL <https://aclanthology.org/2026.acl-long.391/>.
- Peng Xia, Rujun Han, Zifeng Wang, Yanfei Chen, Yufan Zhang, Yoonho Lee, Chengsong Huang, Han Yu, Zhongying CuiZhu, Yifei Ming, Huaxiu Yao, Burak Gokturk, Tomas Pfister, and Chen-Yu Lee. RRSI: Regularized recursive self-improvement of agent harnesses, 2026. URL <https://arxiv.org/abs/2609.24972v1>. Version 1.
- Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-Mem: Agentic memory for LLM agents. In *Advances in Neural Information Processing Systems*, volume 38, 2025. URL https://proceedings.neurips.cc/paper_files/paper/2025/hash/19909c36f51abc4856b4560aff3d36d6-Abstract-Conference.html.
- Yisen Xu, Jiayuan Zhou, Ruiqi Pan, and Tse-Hsun Chen. Reusing past repairs through hierarchical trajectory abstraction for coding agents, 2026. URL <https://arxiv.org/abs/2607.29658v1>. Version 1.
- John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems*, volume 37, 2024. doi: 10.52202/079017-1601. URL https://proceedings.neurips.cc/paper_files/paper/

2024/hash/5a7c947568c1b1328ccc5230172e1e7c-Abstract-Conference.html.

Xunjian Yin, Xinyi Wang, Liangming Pan, Li Lin, Xiaojun Wan, and William Yang Wang. Gödel Agent: A self-referential agent framework for recursively self-improvement. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 27890–27913, 2025. doi: 10.18653/v1/2025.acl-long.1354. URL <https://aclanthology.org/2025.acl-long.1354/>.

Jenny Zhang, Shengran Hu, Cong Lu, Robert Lange, and Jeff Clune. Darwin Gödel Machine: Open-ended evolution of self-improving agents. In *International Conference on Learning Representations*, 2026a. URL <https://openreview.net/forum?id=pUpzQZTvGY>.

Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, Fenglu Hong, Vamsidhar Kamamuru, Jay Rainton, Chen Wu, Mengmeng Ji, Hanchen Li, Urmish Thakker, James Zou, and Kunle Olukotun. Agentic context engineering: Evolving contexts for self-improving language models. In *International Conference on Learning Representations*, 2026b. URL <https://arxiv.org/abs/2510.04618v3>.

Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. ExpeL: LLM agents are experiential learners. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, 2024. doi: 10.1609/aaai.v38i17.29936. URL <https://ojs.aaai.org/index.php/AAAI/article/view/29936>.

Yujun Zhou, Kehan Guo, Haomin Zhuang, Xiangqi Wang, Yue Huang, Zhenwen Liang, Pin-Yu Chen, Tian Gao, Nuno Moniz, Nitesh V. Chawla, and Xiangliang Zhang. Getting better at working with you: Compiling user corrections into runtime enforcement for coding agents, 2026. URL <https://arxiv.org/abs/2606.13174v1>.

Jiayuan Zhu, Junde Wu, Minhao Hu, Shengda Zhu, Jiazhen Pan, Weixiang Shen, Yijun Yang, Fenglin Liu, Jianye Hao, Yueming Jin, Qirong Ho, and Min Xu. Swe context bench: A benchmark for context learning in coding, 2026. URL <https://arxiv.org/abs/2602.08316v3>. Version 3.

A MAIN BENCHMARK COMPARISONS

Table 9: Repair resolution rates (%). Panel A tabulates Figure 3; the LoLBench comparison is in Table 15. Panel B groups published reference results by their reported configuration. Bold marks the best result per benchmark in Panel A. An em dash indicates that the cited source does not report the corresponding benchmark.

A. Three-benchmark comparison

Method	Verified500	Related-Lite99	DeepSWE113
No memory	61.6	21.4	60.8
FAISS Flat	65.2	26.8	64.6
ExpeL	62.4	20.7	58.7
ExpeRepair [†]	63.6	31.3	64.4
ReasoningBank	67.0	24.9	61.7
ACE [†]	62.2	28.1	62.8
Supermemory	63.6	26.5	66.2
ContextGraph	69.2	35.1	72.1

Verified500, Related-Lite99, and DeepSWE113 use DeepSeek V4 Pro 0813 and mini-SWE-agent. Related-Lite99 and DeepSWE113 rates are means across five runs, each reset to the same initial memory pool, with tasks evaluated in issue creation-time order; per-run rates use completed evaluations as the denominator. ContextGraph adds lessons from completed successful and failed attempts to memory for subsequent retrieval; baselines use their open-source defaults for memory updates. The initial Verified500 memory draws on Nebius SWE-bench-extra trajectories, which contain none of the evaluated instances; the DeepSWE113 memory draws on trajectories from LoLBench repositories (Appendix B). [†] ExpeRepair and ACE are our adaptations to mini-SWE-agent. Method sources: ExpeL (Zhao et al., 2024), ExpeRepair (Mu et al., 2026), ReasoningBank (Ouyang et al., 2026), ACE (Zhang et al., 2026b), and Supermemory (Daga et al., 2026).

B. Published reference results, grouped by their reported configuration

Method	Backbone	Verified500 (%)	Related-Lite99 (%)	Source
ExpeRepair	Claude 3.5 Sonnet	57.2	—	ER
ExpeRepair	Claude 4 Sonnet	74.6	—	ER
ReasoningBank	Gemini 2.5 Flash	38.8	—	RB
ReasoningBank	Gemini 2.5 Pro	57.4	—	RB
No memory	Claude Sonnet 4.5	—	26.26	SC
Supermemory	Claude Sonnet 4.5	—	30.30	SC
Oracle summary	Claude Sonnet 4.5	—	34.34	SC

ER: ExpeRepair, Table 1; RB: ReasoningBank, Table 2; SC: SWE-ContextBench, Table 4. ER uses its own repair pipeline with occasional o4-mini fallback; RB uses mini-SWE-agent; SC uses Claude Code. Oracle summaries supply the known related history. Published scores use their original budgets and memory settings; Appendix F reproduces the full published SWE-ContextBench Lite table.

B OVERLAP AUDIT OF INITIAL MEMORY POOLS

Every initial memory pool of the main comparison (Table 2) was audited before evaluation so that no evaluated task, its reference patch, or a near duplicate of that patch can be retrieved. The audit has two steps and was applied to each of these pools in the same way. The curated episodic pool of the retry experiment (Section 5.4) predates this audit; that experiment therefore also reports its result with the ten near-duplicate targets removed.

Instance-level exclusion. Each source trajectory carries its task identifier, repository, and base commit. Trajectories whose identifier matches an evaluated task are removed. The Nebius SWE-bench-extra trajectories that seed the Verified500 pool come from a different task collection, and none of the 500 SWE-bench Verified evaluation instances appears among them, so the evaluated issues, their reference patches, and their hidden tests are absent from memory. The Related-Lite99 pool is the SWE-bench cold-start pool of the SWE-ContextBench protocol, which grows task by task during evaluation; its initial trajectories contain none of the 99 targets. The DeepSWE113 pool contains only LoLBench repository trajectories; no DeepSWE task or repository snapshot appears in it, and the LoLBench and DeepSWE113 repository sets are disjoint. The LoLBench pool excludes the 20 evaluated Python tasks.

Patch-similarity exclusion. Instance identifiers do not catch the same fix recorded under a different task. We therefore compare the reference patch of every evaluated task with every patch in the pool using token-Jaccard similarity, the criterion that identified ten near-duplicate sources in the curated episodic pool (Section 5.4). Pool entries with similarity of at least 0.85 to any evaluated task’s reference patch were removed before the main comparison. Table 10 summarizes the audit for each pool: the instance check removed nothing, and the patch-similarity check removed between 3 and 18 entries per pool, at most 1.3% of any pool.

Table 10: Overlap audit of the initial memory pools used in the main comparison (sources as in Table 2). Counts are source trajectories. Instance match removes trajectories whose task identifier is an evaluated task; patch similarity removes trajectories whose patch has token-Jaccard similarity of at least 0.85 to an evaluated task’s reference patch.

Pool	Source trajectories	Instance match	Patch similarity	Remaining
Verified500	1,762	0	18	1,744
Related-Lite99	1,500	0	6	1,494
DeepSWE113	500	0	3	497
LoLBench Python	400	0	5	395

Retrieval-time check. The selection logs of the main comparison record the source task of every lesson the agent selects; after the audit above, no selected lesson can originate from an evaluated task, and Section 5.5 uses the same logs to report how many originate from a different repository than the target. Official benchmark test results are never part of the agent’s context or stored memory, including between repeated attempts (Section 5.1).

C ADDITIONAL SYSTEM COMPARISONS

These comparisons examine summary and episodic memory across models and attempt budgets. Two configurations of ContextGraph appear in them and are named explicitly here. The *summary-memory configuration* is the experience graph of Section 4 built from an initial pool of 300 experience summaries distilled from the audited SWE-bench cold-start pool (Appendix B), with the mini-SWE-agent scaffold and the main-comparison budget; it is the Related-Lite99 run of the main comparison, which uses the experience graph on all three benchmarks. Figure 4 and Table 8 use it to examine other backbones and initial pool sizes. The *episodic-memory configuration* serves 291 curated repair records with prior code history, reference fixes, and verification hints; Table 5 and Table 11 use it. Table 12 gives an early 50-task development comparison on SWE-bench Verified. Resolution rates are percentages; gains marked pp are percentage-point differences from no memory.

Memory access and evaluation. The agent uses its full current context to search the experience pool, read records, and follow links to relevant strategies. In the backbone comparison, the Kimi K3 and MiniMax M2.7 runs use one attempt per task and report rates over completed evaluations; MiniMax M2.7 uses an API compatibility layer. The pool-size study varies the initial pool from 50 to 300 summaries (Table 8). In the curated-episode comparison, each pass adds the distilled lessons of its completed attempts to the pool before the next pass retries unresolved tasks (Section 5.4). Official test results are withheld from the agent throughout, including between attempts.

Source-similarity sensitivity. A separate analysis removes ten targets whose source fixes have token-Jaccard similarity of at least 0.85. These are distinct tasks, not target trajectories in the memory pool. On the remaining 89 tasks, DeepSeek V4 Pro resolves 21.35% with memory versus 14.61% without memory ($p = .031$). Appendix B applies the same criterion to every initial pool of the main comparison.

Table 11: Curated-episode Related-Lite99 replication, one pass per backbone. Resolution rates are percentages over completed evaluations. The last column gives no-memory/ContextGraph rates on the fixed 89-task subset after removing ten near-duplicate targets, followed by the paired p value. N/C only counts tasks resolved exclusively by no memory/ContextGraph.

Backbone	No memory	ContextGraph	N/C only	p	89-task subset
DeepSeek V4 Pro	14.14	21.21	0/7	.016	14.61/21.35 ($p = .031$)
DeepSeek V3.2 exp	16.33	20.20	4/8	.39	16.85/20.22 ($p = .55$)
GLM 5.2	17.17	24.24	1/8	.039	16.85/23.60 ($p = .070$)
Kimi K3	29.29	31.31	3/5	.73	29.21/31.46 ($p = .73$)

Table 12: Separate 50-task development comparison. Completion uses all 50 tasks; resolution uses completed evaluations. The recorded 66.6% retains its original display precision.

Method	Completion (%)	Resolution (%)
ContextGraph	96.0	66.6
ExpeL	100.0	64.0
FAISS Flat	98.0	61.2
No memory	100.0	58.0

Table 13: Experience-graph snapshot, using text-embedding-3-large vectors with 3,072 dimensions. The seven typed relations are listed in Section 4.1.

Node label	Count	Role
Trajectory	1,795	Source episode
Fragment	13,813	Action and observation slice
ErrorPattern	586	Error type and keywords
Strategy	9,600	Extracted rule
CanonicalRule	7,835	Consolidated rule
PlaybookEntry	8,910	Retrieval-ready entry
Community	781	Linked-fragment summary
ProblemSummary	1,795	Episode problem abstraction
Total nodes	45,115	
Total edges	73,975	Seven typed relations

D ADDITIONAL BENCHMARK AND AGENT RESULTS

This appendix tabulates the DeepSeek V4 Pro column of Figure 4 (Table 14) and gives the LoLBench comparison from Section 5.2. Gemini results appear in Table 3. The three-benchmark comparison is tabulated in Table 9; additional comparisons appear in Appendix C.

Table 14: DeepSeek V4 Pro column of Figure 4 (300-summary initial pool; the ContextGraph and no-memory values are those of the main comparison). Bold marks the best automatic memory result; the oracle receives a known related summary.

Method	Resolution rate (%)
No memory	21.4
Random summaries	27.1
Mem0	26.8
LangMem	34.0
ContextGraph	35.1
Oracle summary	27.8

LoLBench Python subset. We select 20 Python tasks from LoLBench (Peng, 2026), which evaluates long-horizon development from enhancement proposals in large software systems. The agent is Codex with GPT-5.5. The initial experience graph is built from other Python tasks, separate from the 20 evaluation tasks. During evaluation, ContextGraph adds lessons from completed attempts, including successes and failures, for subsequent retrieval. Baseline memory updates follow each method’s open-source defaults.

Table 15: Success rates on 20 LoLBench Python tasks with Codex and GPT-5.5, as reported in Section 5.2. $\text{pass}@k$ reports the fraction of tasks solved within k attempts.

Method	Metric	Success rate (%)
No memory	pass@1	5.0
FAISS Flat	pass@1	5.0
ExpeL	pass@1	5.0
ExpeRepair	pass@1	5.0
ReasoningBank	pass@1	5.0
ACE	pass@1	5.0
Supermemory	pass@1	5.0
ContextGraph	pass@1	15.0
ContextGraph	pass@3	25.0

E FULL-CONTEXT AGENT-DIRECTED MEMORY SELECTION

At step t , the agent uses the complete context C_t : the issue and conversation, inspected code, edits, actions, and observed failures. It can search the whole experience pool, read source records, follow links between strategies and errors, and compare lessons across repositories. Its selection follows Equation 2: the agent decides which records to inspect and retain, without a fixed relevance score or number of nearest neighbours. Each search or read adds its results to H_j before the next memory action; the selected lessons form $\mathcal{M}_t$.

The selected memory contains strategies and warnings, their source context, and the conditions under which they are useful. Before planning, selection helps choose an investigation strategy. After an error, the updated context helps revise the diagnosis and find additional guidance. After each completed task, the graph incorporates new experience for subsequent tasks. The pool therefore evolves along the issue-ordered task stream; official benchmark test results are never part of the agent’s context or stored memory.

Algorithm 1 ContextGraph across a stream of coding tasks

Input: initial trajectories H , coding tasks $q_1, \dots, q_T$.

Output: candidate patches and an updated experience graph G .

1. Extract strategies and warnings from H ; build G and its retrieval entries.
 2. **For each** task q_t in issue creation-time order:
 - a. Use the full task context to search G and select guidance for planning.
 - b. Execute investigation, editing, and testing actions; record the trajectory.
 - c. After an error, search again with the updated full context; revise the plan.
 - d. Continue the repair loop until the task ends or its budget is exhausted.
 - e. Distill experience from the trajectory and agent-visible outcomes.
 - f. Update G and retrieval entries; expose the new experience to later tasks.
-

F PUBLISHED SWE-CONTEXTBENCH REFERENCE RESULTS

Table 16 reproduces the resolved rates from Table 4 of Zhu et al. (2026). Supermemory leads the automatic memory settings reported there; oracle summaries use known task relationships to supply the relevant history. These published results provide context for our Related-Lite experiments. Direct comparisons require matching task IDs, backbone, agent, and attempt budget.

Table 16: Published SWE-ContextBench Lite results: 99 related tasks, Claude Sonnet 4.5. Rates are copied as reported, including OpenViking’s 29.20%. Δ is the percentage-point change from no context. Oracle rows are separated from automatic retrieval.

Setting	History access	Resolved (%)	Δ (pp)
No context	None	26.26	0.00
Free context	Agent-selected trajectories	26.26	0.00
Free summary	Agent-selected summaries	22.22	−4.04
Mem0	Automatic retrieval	24.24	−2.02
OpenViking	Automatic retrieval	29.20	+2.94
Supermemory	Automatic retrieval	30.30	+4.04
Oracle context	Supplied related trajectory	27.27	+1.01
Oracle summary	Supplied related summary	34.34	+8.08

On the separate 376-task set, Table 3 reports its best resolution rate of 23.94% for GPT-5.3 Codex with oracle summaries. Supermemory’s 55.95% in Table 4 is a test-level FAIL_TO_PASS rate, not task resolution. LangMem has retrieval-quality results in Table 5 but no resolved rate in Table 4.

LIMITATIONS

The main comparisons measure full memory systems; the ablation in Section 5.3 separates their components on one benchmark, the task-stream study uses the retry protocol with a separate curated pool and budget, and the cross-repository analysis covers the main protocol only. The 20-task LoLBench study covers one language and a small part of that benchmark. Longer task streams and additional agent scaffolds would clarify when each design contributes most. Detailed settings and additional results accompany the appendices.

REPRODUCIBILITY STATEMENT

The anonymous repository (Section 5.1) releases the code, extraction prompts, graph schema, initial memory pools, evaluation scripts, and the scripts and data that generate the figures (`figures/scripts`). The main text defines the memory workflow, benchmark settings, and reported metrics. The appendices provide the overlap audit of every initial memory pool of the main comparison, numerical comparisons, and supplementary implementation details. ContextGraph’s main comparisons include online experience updates; baseline updating follows the corresponding open-source implementation.

ETHICS STATEMENT

The reported evaluation uses software-engineering benchmarks. No confidential enterprise scenarios or data are included. Retrieved strategies and warnings guide the agent’s investigation; patches are evaluated through the benchmark harnesses.

AI USE STATEMENT

Generative AI assisted literature review, method development, implementation, experiments, analysis, and writing, including drafting and revising the abstract.